Speedup Completion Guide for DAMPS-MMHCL

Closing the Remaining Gaps to 100% Compliance with the
DAMPS_MMHCL_Training_Speedup_Guide_EN

Overall Verdict

The current implementation does **not** yet reach 100% compliance. Based on a direct inspection of the GitHub repository and the attached Jupyter notebook, the runtime-level compliance with the *DAMPS_MMHCL_Training_Speedup_Guide_EN* sits at approximately **85–90%**.

1 What Has Been Implemented Correctly

The following speedup items from the guide are already wired in correctly and are visible in the latest commit of the repository:

- ✓ **cuFFT plan-cache lockdown**
  `torch.backends.cuda.cufft_plan_cache[idx].max_size = 1` is set at startup, preventing plan-cache memory leaks.
- ✓ **Orthonormal rFFT/irFFT**
  `torch.fft.rfft` and `torch.fft.irfft` are invoked with `norm="ortho"`, exactly as recommended for the d=64 embeddings.
- ✓ **Dual-path KNN with FAISS-GPU fallback**
  `DualPathKNN` is parameterised with both `faiss_threshold` and `faiss_use_gpu` arguments inside `train.py`.
- ✓ **`torch.compile` on the DAMPS submodule**
  The forward path of the DAMPS module is compiled using `mode="reduce-overhead"` and `dynamic=True`, while the K-NN rebuild step is correctly left uncompiled (matching the caveat in the speedup guide).
- ✓ **bfloat16 mixed precision via `torch.autocast`**
  The training loop uses `torch.amp.autocast(device_type=..., dtype=torch.bfloat16, enabled=use_amp)` without a `GradScaler`, which is the correct pattern for bf16.
- ✓ **Optuna TPE Bayesian HPO**
  `scripts/run_optuna_hpo.py` is present and ready to be invoked.
- ✓ **wandb Sweeps integration**
  Both `scripts/run_wandb_sweep.py` and `scripts/wandb_sweep.yaml` exist in the repository.

- ✓ **Paired t-test via `scipy.stats.ttest_rel`**
  Implemented in `scripts/paired_ttest.py`.
- ✓ **Permutation-FFT aggregator**
  `scripts/_aggregate_permutation_fft.py` is in place, paired with notebook cell 5.5.

2 Three Blocking Issues Preventing Full Compliance

2.1 The notebook does not enable `torch.compile`

`train.py` already exposes the command-line flags `--torch_compile_mode` and `--torch_compile_dynamic`, but the notebook does not pass them into the subprocess command list. As a result, the 25–35% forward-pass speedup from `torch.compile` is **not actually activated at runtime**, even though the supporting code is fully present.

Fix. Append the following two key/value pairs to the `cmd` list that launches `train.py` from the notebook:

```
"--torch_compile_mode", "reduce-overhead",
"--torch_compile_dynamic", "1",
```

2.2 The notebook does not pass `--faiss_use_gpu 1`

The notebook currently passes only `--faiss_threshold 60000`, but never `--faiss_use_gpu 1`. For the Amazon Clothing dataset (N=23,033 < 60K) this has no practical impact, because the chunked PyTorch path is selected anyway. However, when scaling up to a larger dataset, the FAISS-GPU fallback will silently remain disabled.

Fix. Add the GPU FAISS flag to the notebook `cmd` list as well:

```
"--faiss_use_gpu", "1",
```

2.3 `n_runs = 3` instead of `10`

The notebook is currently configured with `n_runs = 3`, which violates the statistical-reliability requirement shared by both the architecture spec (§4) and the Speedup Guide (§8): final results must be reported across **10 seeds**, with confidence intervals and paired t-tests.

Fix. Update the notebook constant to:

```
n_runs: int = 10
```

3 Optional Items (Safe to Skip)

- ◆ **`torch_geometric.nn.HypergraphConv` (§3)** — The repository still uses a custom HGCN implementation. Adopting the PyG operator could unlock an additional 25–35% forward-pass speedup, but skipping it does *not* violate the architecture spec.
- ◆ **`einops` (§10)** — A pure readability improvement; it has no measurable effect on runtime and can safely be omitted.

- ◆ **`scipy.stats.vonmises` (§5)** — Not used. The current GPU-vectorised implementation $\hat{\theta}_c = \text{atan2}(\sum \sin R_i, \sum \cos R_i)$ is in fact *faster* than the CPU off-loaded path suggested by the guide, because it avoids CPU–GPU synchronisation. This deviation is a legitimate, beneficial improvement.

4 Compliance Summary

Speedup Guide Item	**Status**
cuFFT plan-cache lock	✓
`torch.fft.rfft` / `irfft` (`norm="ortho"`)	✓
Dual-path KNN + FAISS-GPU (code level)	✓
Dual-path KNN + FAISS-GPU (notebook flag)	▲
`torch.compile` on DAMPS / HGCN (code level)	✓
`torch.compile` (notebook flag)	▲
PyG `HypergraphConv`	✗ (optional)
von Mises `atan2`-based MLE	✓ (better than guide)
Optuna TPE HPO	✓
wandb Sweeps	✓
`scipy.stats.ttest_rel`	✓
bfloat16 `torch.autocast` (no `GradScaler`)	✓
`einops`	✗ (optional)
Permutation-FFT ablation	✓
10 seeds + CI + paired t-test	▲

5 Conclusion

Once the three blocking issues above are patched — and most importantly, once `torch.compile` is actually activated from the notebook — the implementation will reach **full runtime-level compliance** with the *DAMPS_MMHCL_Training_Speedup_Guide_EN*.